

SMART CANTEEN MANAGEMENT SYSTEM

Complete Database Schema, Tables Directory & SQL DDL Specification

Database Name:	canteen_db	Default Engine:	InnoDB (Full ACID Support)
Total Tables:	12 Normalized Tables (3NF Compliant)	Total Views:	4 Analytical & KDS Views
Character Set:	utf8mb4 (Full Unicode / Multilingual)	Triggers & Logic:	Stock Deductions & Price Auditing

PART 1: Master Directory of All 12 Tables

#	Table Name	Primary Key	Foreign Keys (Parent)	Primary Function in System
1	students	student_id	None	Student accounts, Google login, campus wallet balance
2	categories	category_id	None	Food menu categories (South Indian, Snacks, Beverages)
3	menu_items	item_id	category_id -> categories	Food catalog, prices, prep time, availability toggle
4	ingredients	ingredient_id	None	Raw materials kitchen stock, threshold alert levels
5	item_ingredients	(item_id, ingredient_id)	item_id, ingredient_id	Bill of Materials (BOM) recipe composition (M:N link)
6	orders	order_id	student_id -> students	Official Token Numbers, cooking status, order date
7	order_items	order_item_id	order_id, item_id	Dishes contained in an order, quantity, subtotal
8	payments	payment_id	order_id -> orders	Payment receipts, transaction refs (Wallet, UPI, Card)
9	kitchen_staff	staff_id	None	Kitchen staff logins, chef roles (Head Chef, Cook)
10	feedback	feedback_id	order_id, student_id	Student reviews and ratings (1 to 5 stars)
11	inventory_logs	log_id	ingredient_id -> ingredients	Audit log of all stock deductions and restock events
12	price_audit_log	audit_id	item_id -> menu_items	Trigger audit log recording all dish price changes

PART 2: Detailed Table Columns & Constraints

1. Table: students

Stores registered campus students, authentication credentials, and digital campus wallet balance.

Column Name	Data Type	Null / Default	Constraints & Key	Description
student_id	INT	NOT NULL, AUTO_INCREMENT	PRIMARY KEY	Unique auto-generated student identifier
reg_no	VARCHAR(20)	NOT NULL	UNIQUE	Campus student registration number (e.g. 24BCY10379)
full_name	VARCHAR(100)	NOT NULL	-	Student's full legal name
email	VARCHAR(100)	NOT NULL	UNIQUE	Campus Google account email used for portal authentication
phone	VARCHAR(15)	NOT NULL	-	Contact phone number for notifications
wallet_balance	DECIMAL(10,2)	NOT NULL DEFAULT 500.00	CHECK (wallet_balance >= 0)	Digital campus wallet balance in Rupees
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-	Account registration timestamp

2. Table: categories

Organizes the canteen menu catalog into logical dining categories.

Column Name	Data Type	Null / Default	Constraints & Key	Description
category_id	INT	NOT NULL, AUTO_INCREMENT	PRIMARY KEY	Unique category identifier
category_name	VARCHAR(50)	NOT NULL	UNIQUE	Name of category (South Indian, North Indian, Beverages)
description	VARCHAR(255)	NULL	-	Brief summary of category cuisine

3. Table: menu_items

Stores canteen food dishes, selling prices, prep times, and live availability toggles.

Column Name	Data Type	Null / Default	Constraints & Key	Description
item_id	INT	NOT NULL, AUTO_INCREMENT	PRIMARY KEY	Unique dish identifier
category_id	INT	NOT NULL	FK -> categories (CASCADE)	Category classification for this dish
item_name	VARCHAR(100)	NOT NULL	UNIQUE	Name of food dish (e.g. Masala Dosa, Mango Lassi)
description	TEXT	NULL	-	Detailed description of dish ingredients and preparation
price	DECIMAL(8,2)	NOT NULL	CHECK (price > 0)	Selling price in Rupees
preparation_time_mins	INT	DEFAULT 10	CHECK (prep_time >= 0)	Estimated kitchen prep time in minutes
is_available	BOOLEAN	NOT NULL DEFAULT TRUE	-	1 = Active on student menu; 0 = Marked Sold Out
image_url	VARCHAR(255)	NULL	-	Path to dish display image
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-	Item creation timestamp

4. Table: ingredients

Tracks raw grocery items in the kitchen store, current quantities, and restock alert thresholds.

Column Name	Data Type	Null / Default	Constraints & Key	Description
ingredient_id	INT	NOT NULL, AUTO_INCREMENT	PRIMARY KEY	Unique raw ingredient identifier
ingredient_name	VARCHAR(100)	NOT NULL	UNIQUE	Name of raw grocery (Rice Batter, Coffee Beans, Paneer)
unit	VARCHAR(20)	NOT NULL	-	Measurement unit: 'grams', 'ml', 'pieces'
current_stock	DECIMAL(10,2)	NOT NULL DEFAULT 0.00	CHECK (current_stock >= 0)	Current physical stock in kitchen store
min_threshold	DECIMAL(10,2)	NOT NULL DEFAULT 100.00	CHECK (min_threshold >= 0)	Minimum stock limit before low-stock alert triggers
cost_per_unit	DECIMAL(8,2)	NOT NULL DEFAULT 0.00	-	Purchase cost per unit for inventory accounting
last_restocked	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	-	Timestamp of most recent restock event

5. Table: item_ingredients (Bill of Materials - BOM)

Resolves Many-to-Many (M:N) relationship between dishes and raw materials for automated stock deduction.

Column Name	Data Type	Null / Default	Constraints & Key	Description
item_id	INT	NOT NULL	PK (Part 1), FK -> menu_items (CASCADE)	Menu dish ID
ingredient_id	INT	NOT NULL	PK (Part 2), FK -> ingredients (RESTRICT)	Required raw grocery ingredient ID
quantity_required	DECIMAL(8,2)	NOT NULL	CHECK (quantity_required > 0)	Exact quantity of ingredient consumed per 1 serving

6. Table: orders

Central order record storing official Token Numbers, student references, and live cooking pipeline status.

Column Name	Data Type	Null / Default	Constraints & Key	Description
order_id	INT	NOT NULL, AUTO_INCREMENT	PRIMARY KEY (Token #)	Auto-increment token number displayed on KDS
student_id	INT	NOT NULL	FK -> students (RESTRICT)	Student who placed and paid for the order
order_date	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	INDEX idx_orders_date_status	Timestamp when order was submitted
status	ENUM	NOT NULL DEFAULT 'Placed'	'Placed','Preparing','Ready', 'Picked Up','Cancelled'	Current live workflow status in kitchen pipeline
total_amount	DECIMAL(10,2)	NOT NULL DEFAULT 0.00	CHECK (total_amount >= 0)	Total amount charged for the order in Rupees
special_instructions	VARCHAR(255)	NULL	-	Custom notes for the chef (e.g. 'Less spicy, extra chutney')

7. Table: order_items

Line items table storing individual dishes and quantities included within an order.

Column Name	Data Type	Null / Default	Constraints & Key	Description
order_item_id	INT	NOT NULL, AUTO_INCREMENT	PRIMARY KEY	Unique line item identifier
order_id	INT	NOT NULL	FK -> orders (CASCADE)	Reference to parent order / Token Number
item_id	INT	NOT NULL	FK -> menu_items (RESTRICT)	Reference to the ordered menu dish
quantity	INT	NOT NULL DEFAULT 1	CHECK (quantity > 0)	Number of portions ordered
unit_price	DECIMAL(8,2)	NOT NULL	-	Locked-in unit price at the time of purchase
subtotal	DECIMAL(10,2)	NOT NULL	CHECK (subtotal >= 0)	Total line cost = quantity * unit_price

8. Table: payments

Payment audit log linking 1-to-1 with orders to verify transaction references and payment channels.

Column Name	Data Type	Null / Default	Constraints & Key	Description
payment_id	INT	NOT NULL, AUTO_INCREMENT	PRIMARY KEY	Unique payment transaction identifier
order_id	INT	NOT NULL	UNIQUE, FK -> orders (CASCADE)	1-to-1 link to the corresponding order
payment_method	ENUM	NOT NULL DEFAULT 'UPI'	'UPI', 'Card', 'Cash', 'Student Wallet'	Payment channel chosen by student
payment_status	ENUM	NOT NULL DEFAULT 'Completed'	'Pending', 'Completed', 'Failed', 'Refunded'	Current status of financial transaction
amount	DECIMAL(10,2)	NOT NULL	CHECK (amount >= 0)	Total amount collected in Rupees
transaction_ref	VARCHAR(64)	NOT NULL	UNIQUE	Unique transaction reference code
payment_time	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-	Timestamp of successful payment capture

9. Table: kitchen_staff

Staff registry for kitchen and canteen personnel managing the Kitchen Display System (KDS).

Column Name	Data Type	Null / Default	Constraints & Key	Description
staff_id	INT	NOT NULL, AUTO_INCREMENT	PRIMARY KEY	Unique staff member identifier
full_name	VARCHAR(100)	NOT NULL	-	Staff member legal name
email	VARCHAR(100)	NOT NULL	UNIQUE	Staff login email authenticated against database
role	ENUM	NOT NULL DEFAULT 'Cook'	'Head Chef', 'Cook', 'Canteen Manager'	Role-based access permissions
is_active	BOOLEAN	NOT NULL DEFAULT TRUE	-	1 = Active staff member; 0 = Deactivated account
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-	Account creation timestamp

10. Table: feedback

Customer quality ratings and review comments submitted by students upon meal completion.

Column Name	Data Type	Null / Default	Constraints & Key	Description
feedback_id	INT	NOT NULL, AUTO_INCREMENT	PRIMARY KEY	Unique review identifier
order_id	INT	NOT NULL	FK -> orders (CASCADE)	Reference to the fulfilled order
student_id	INT	NOT NULL	FK -> students (CASCADE)	Student who submitted the review
rating	INT	NOT NULL	CHECK (rating BETWEEN 1 AND 5)	Numerical satisfaction rating from 1 to 5 stars
comments	TEXT	NULL	-	Optional student comments on food taste, hygiene, or speed
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-	Submission timestamp

11. Table: inventory_logs

Immutable audit trail logging every stock deduction and restock transaction in the kitchen store.

Column Name	Data Type	Null / Default	Constraints & Key	Description
log_id	INT	NOT NULL, AUTO_INCREMENT	PRIMARY KEY	Unique stock log identifier
ingredient_id	INT	NOT NULL	FK -> ingredients (CASCADE)	Ingredient whose quantity was changed
change_type	ENUM	NOT NULL	'DEDUCTION_ORDER', 'RESTOCK_MANUAL', 'ADJUSTMENT'	Reason for inventory stock change
quantity_changed	DECIMAL(10,2)	NOT NULL	-	Amount deducted (-) or added (+) in ingredient units
remaining_stock	DECIMAL(10,2)	NOT NULL	-	Exact balance remaining after the change occurred
reference_order_id	INT	NULL	-	Associated Token Number if deduction was triggered by an order
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-	Exact timestamp of stock change

12. Table: price_audit_log

Audit table populated automatically by trigger whenever a dish price is altered.

Column Name	Data Type	Null / Default	Constraints & Key	Description
audit_id	INT	NOT NULL, AUTO_INCREMENT	PRIMARY KEY	Unique audit record identifier
item_id	INT	NOT NULL	-	Menu dish ID whose price was modified
old_price	DECIMAL(8,2)	NOT NULL	-	Previous selling price in Rupees
new_price	DECIMAL(8,2)	NOT NULL	-	New updated selling price in Rupees
changed_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-	Timestamp when price update was committed
changed_by	VARCHAR(50)	DEFAULT 'ADMIN'	-	Database user session that executed the UPDATE

PART 3: Database Views, Triggers & Procedures Directory

The system defines 4 Views, 2 Automated Triggers, and 3 Stored Procedures/Functions:

View Name	Underlying Tables	Business Function & Query Logic
vw_live_menu	menu_items, categories, item_ingredients, ingredients	Computes real-time stock availability for every dish. If any ingredient required in the recipe has depleted, automatically flags dish as 'Out of Stock'.
vw_kitchen_queue	orders, students, order_items, menu_items	Powers the Kitchen Display System (KDS). Joins orders with students and groups order items into a single string summary (e.g. '2x Masala Dosa, 1x Mango Lassi').
vw_low_stock_alerts	ingredients	Filters inventory items where current_stock <= min_threshold. Calculates exact deficit amount and assigns urgency level ('WARNING', 'HIGH', 'CRITICAL').
vw_daily_sales_summary	orders	Aggregates daily sales performance: total orders count, gross sales volume, fulfilled revenue, and average ticket size.

Component Type	Object Name	Event / Signature	Description
TRIGGER	trg_check_and_deduct_stock	AFTER INSERT ON order_items	Automatically deducts recipe ingredients from stock; aborts order if ingredients are insufficient.
TRIGGER	trg_audit_price_change	BEFORE UPDATE ON menu_items	Logs old price, new price, user, and timestamp into price_audit_log upon price alteration.
PROCEDURE	sp_update_order_status	(p_order_id, p_new_status)	Advances order status safely along cooking pipeline (Placed -> Preparing -> Ready -> Picked Up).
PROCEDURE	sp_restock_ingredient	(p_ingredient_id, p_quantity)	Increases stock for specified raw grocery material and records transaction in inventory_logs.
FUNCTION	fn_calculate_bill	(p_subtotal, p_tax_rate)	Calculates final bill amount including campus facility charge / tax percentage.

PART 4: Complete SQL DDL Schema Code (Create Table Statements)

Below are the exact, complete, copy-pasteable MySQL 8.0 DDL statements for creating the entire database schema:

Database Initialization

```
CREATE DATABASE IF NOT EXISTS canteen_db;
USE canteen_db;
```

1. Table: students

```
CREATE TABLE students (
  student_id INT AUTO_INCREMENT PRIMARY KEY,
  reg_no VARCHAR(20) NOT NULL UNIQUE,
  full_name VARCHAR(100) NOT NULL,
  email VARCHAR(100) NOT NULL UNIQUE,
  phone VARCHAR(15) NOT NULL,
  wallet_balance DECIMAL(10, 2) NOT NULL DEFAULT 500.00,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT chk_wallet_positive CHECK (wallet_balance >= 0)
) ENGINE=InnoDB;
```

2. Table: categories

```
CREATE TABLE categories (
  category_id INT AUTO_INCREMENT PRIMARY KEY,
  category_name VARCHAR(50) NOT NULL UNIQUE,
  description VARCHAR(255)
) ENGINE=InnoDB;
```

3. Table: menu_items

```
CREATE TABLE menu_items (
  item_id INT AUTO_INCREMENT PRIMARY KEY,
  category_id INT NOT NULL,
  item_name VARCHAR(100) NOT NULL UNIQUE,
  description TEXT,
  price DECIMAL(8, 2) NOT NULL,
  preparation_time_mins INT DEFAULT 10,
  is_available BOOLEAN NOT NULL DEFAULT TRUE,
  image_url VARCHAR(255),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_menu_category FOREIGN KEY (category_id)
    REFERENCES categories(category_id) ON DELETE CASCADE,
  CONSTRAINT chk_item_price CHECK (price > 0),
  CONSTRAINT chk_prep_time CHECK (preparation_time_mins >= 0)
) ENGINE=InnoDB;
```

4. Table: ingredients (Raw Kitchen Inventory)

```
CREATE TABLE ingredients (
  ingredient_id INT AUTO_INCREMENT PRIMARY KEY,
  ingredient_name VARCHAR(100) NOT NULL UNIQUE,
  unit VARCHAR(20) NOT NULL,
  current_stock DECIMAL(10, 2) NOT NULL DEFAULT 0.00,
  min_threshold DECIMAL(10, 2) NOT NULL DEFAULT 100.00,
  cost_per_unit DECIMAL(8, 2) NOT NULL DEFAULT 0.00,
  last_restocked TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT chk_stock_non_negative CHECK (current_stock >= 0),
  CONSTRAINT chk_threshold_positive CHECK (min_threshold >= 0)
) ENGINE=InnoDB;
```

5. Table: item_ingredients (Bill of Materials - BOM)

```
CREATE TABLE item_ingredients (
  item_id INT NOT NULL,
  ingredient_id INT NOT NULL,
  quantity_required DECIMAL(8, 2) NOT NULL,
  PRIMARY KEY (item_id, ingredient_id),
  CONSTRAINT fk_recipe_item FOREIGN KEY (item_id)
    REFERENCES menu_items(item_id) ON DELETE CASCADE,
  CONSTRAINT fk_recipe_ingredient FOREIGN KEY (ingredient_id)
    REFERENCES ingredients(ingredient_id) ON DELETE RESTRICT,
  CONSTRAINT chk_qty_req CHECK (quantity_required > 0)
) ENGINE=InnoDB;
```

6. Table: orders (Token Number Queue)

```
CREATE TABLE orders (  
  order_id INT AUTO_INCREMENT PRIMARY KEY,  
  student_id INT NOT NULL,  
  order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  status ENUM('Placed', 'Preparing', 'Ready', 'Picked Up', 'Cancelled') NOT NULL DEFAULT 'Placed',  
  total_amount DECIMAL(10, 2) NOT NULL DEFAULT 0.00,  
  special_instructions VARCHAR(255),  
  CONSTRAINT fk_orders_student FOREIGN KEY (student_id)  
    REFERENCES students(student_id) ON DELETE RESTRICT,  
  CONSTRAINT chk_order_total CHECK (total_amount >= 0)  
) ENGINE=InnoDB;
```

7. Table: order_items

```
CREATE TABLE order_items (  
  order_item_id INT AUTO_INCREMENT PRIMARY KEY,  
  order_id INT NOT NULL,  
  item_id INT NOT NULL,  
  quantity INT NOT NULL DEFAULT 1,  
  unit_price DECIMAL(8, 2) NOT NULL,  
  subtotal DECIMAL(10, 2) NOT NULL,  
  CONSTRAINT fk_orderitems_order FOREIGN KEY (order_id)  
    REFERENCES orders(order_id) ON DELETE CASCADE,  
  CONSTRAINT fk_orderitems_item FOREIGN KEY (item_id)  
    REFERENCES menu_items(item_id) ON DELETE RESTRICT,  
  CONSTRAINT chk_order_qty CHECK (quantity > 0),  
  CONSTRAINT chk_subtotal CHECK (subtotal >= 0)  
) ENGINE=InnoDB;
```

8. Table: payments

```
CREATE TABLE payments (  
  payment_id INT AUTO_INCREMENT PRIMARY KEY,  
  order_id INT NOT NULL UNIQUE,  
  payment_method ENUM('UPI', 'Card', 'Cash', 'Student Wallet') NOT NULL DEFAULT 'UPI',  
  payment_status ENUM('Pending', 'Completed', 'Failed', 'Refunded') NOT NULL DEFAULT 'Completed',  
  amount DECIMAL(10, 2) NOT NULL,  
  transaction_ref VARCHAR(64) NOT NULL UNIQUE,  
  payment_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  CONSTRAINT fk_payments_order FOREIGN KEY (order_id)  
    REFERENCES orders(order_id) ON DELETE CASCADE,  
  CONSTRAINT chk_payment_amount CHECK (amount >= 0)  
) ENGINE=InnoDB;
```

9. Table: kitchen_staff

```
CREATE TABLE kitchen_staff (  
  staff_id INT AUTO_INCREMENT PRIMARY KEY,  
  full_name VARCHAR(100) NOT NULL,  
  email VARCHAR(100) NOT NULL UNIQUE,  
  role ENUM('Head Chef', 'Cook', 'Canteen Manager') NOT NULL DEFAULT 'Cook',  
  is_active BOOLEAN NOT NULL DEFAULT TRUE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
) ENGINE=InnoDB;
```

10. Table: feedback

```
CREATE TABLE feedback (  
  feedback_id INT AUTO_INCREMENT PRIMARY KEY,  
  order_id INT NOT NULL,  
  student_id INT NOT NULL,  
  rating INT NOT NULL,  
  comments TEXT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  CONSTRAINT fk_feedback_order FOREIGN KEY (order_id)  
    REFERENCES orders(order_id) ON DELETE CASCADE,  
  CONSTRAINT fk_feedback_student FOREIGN KEY (student_id)  
    REFERENCES students(student_id) ON DELETE CASCADE,  
  CONSTRAINT chk_rating_range CHECK (rating BETWEEN 1 AND 5)  
) ENGINE=InnoDB;
```

11. Table: inventory_logs

```
CREATE TABLE inventory_logs (
  log_id INT AUTO_INCREMENT PRIMARY KEY,
  ingredient_id INT NOT NULL,
  change_type ENUM('DEDUCTION_ORDER', 'RESTOCK_MANUAL', 'ADJUSTMENT') NOT NULL,
  quantity_changed DECIMAL(10, 2) NOT NULL,
  remaining_stock DECIMAL(10, 2) NOT NULL,
  reference_order_id INT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_invlog_ingredient FOREIGN KEY (ingredient_id)
    REFERENCES ingredients(ingredient_id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

12. Table: price_audit_log

```
CREATE TABLE price_audit_log (
  audit_id INT AUTO_INCREMENT PRIMARY KEY,
  item_id INT NOT NULL,
  old_price DECIMAL(8, 2) NOT NULL,
  new_price DECIMAL(8, 2) NOT NULL,
  changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  changed_by VARCHAR(50) DEFAULT 'ADMIN'
) ENGINE=InnoDB;
```

Database Indexes for Optimization

```
CREATE INDEX idx_orders_date_status ON orders(order_date, status);
CREATE INDEX idx_orders_student ON orders(student_id);
CREATE INDEX idx_order_items_item ON order_items(item_id);
CREATE INDEX idx_ingredients_stock ON ingredients(current_stock, min_threshold);
CREATE INDEX idx_menu_category ON menu_items(category_id, is_available);
```

PART 5: SQL DDL for Views & Automated Triggers

View: vw_live_menu

```
CREATE OR REPLACE VIEW vw_live_menu AS
SELECT
  m.item_id, m.item_name, c.category_name, m.description, m.price,
  m.preparation_time_mins, m.is_available,
  CASE
    WHEN MIN(COALESCE(i.current_stock - ii.quantity_required, 0)) >= 0 THEN 'In Stock'
    ELSE 'Out of Stock'
  END AS stock_status
FROM menu_items m
JOIN categories c ON m.category_id = c.category_id
LEFT JOIN item_ingredients ii ON m.item_id = ii.item_id
LEFT JOIN ingredients i ON ii.ingredient_id = i.ingredient_id
GROUP BY m.item_id, m.item_name, c.category_name, m.description, m.price, m.preparation_time_mins, m.is_available;
```

View: vw_kitchen_queue (Kitchen Display System)

```
CREATE OR REPLACE VIEW vw_kitchen_queue AS
SELECT
  o.order_id, s.reg_no, s.full_name AS student_name, o.status,
  o.total_amount, o.special_instructions, o.order_date,
  TIMESTAMPDIFF(MINUTE, o.order_date, NOW()) AS elapsed_minutes,
  GROUP_CONCAT(CONCAT(oi.quantity, 'x ', mi.item_name) SEPARATOR ', ') AS item_summary
FROM orders o
JOIN students s ON o.student_id = s.student_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN menu_items mi ON oi.item_id = mi.item_id
WHERE o.status IN ('Placed', 'Preparing', 'Ready')
GROUP BY o.order_id, s.reg_no, s.full_name, o.status, o.total_amount, o.special_instructions, o.order_date
ORDER BY
  CASE o.status WHEN 'Placed' THEN 1 WHEN 'Preparing' THEN 2 WHEN 'Ready' THEN 3 END,
  o.order_date ASC;
```

Trigger: trg_check_and_deduct_stock (Automated Inventory BOM)

```

DELIMITER //
CREATE TRIGGER trg_check_and_deduct_stock
AFTER INSERT ON order_items
FOR EACH ROW
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE v_ing_id INT;
    DECLARE v_qty_req DECIMAL(8, 2);
    DECLARE v_curr_stock DECIMAL(10, 2);
    DECLARE v_ing_name VARCHAR(100);
    DECLARE v_needed DECIMAL(10, 2);

    DECLARE cur_recipe CURSOR FOR
        SELECT ii.ingredient_id, ii.quantity_required, i.current_stock, i.ingredient_name
        FROM item_ingredients ii
        JOIN ingredients i ON ii.ingredient_id = i.ingredient_id
        WHERE ii.item_id = NEW.item_id;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    OPEN cur_recipe;
    recipe_loop: LOOP
        FETCH cur_recipe INTO v_ing_id, v_qty_req, v_curr_stock, v_ing_name;
        IF done THEN LEAVE recipe_loop; END IF;

        SET v_needed = v_qty_req * NEW.quantity;
        IF v_curr_stock < v_needed THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'INSUFFICIENT_STOCK: Out of required ingredient for order';
        END IF;

        UPDATE ingredients SET current_stock = current_stock - v_needed WHERE ingredient_id = v_ing_id;
        INSERT INTO inventory_logs (ingredient_id, change_type, quantity_changed, remaining_stock, reference_order_id)
        VALUES (v_ing_id, 'DEDUCTION_ORDER', v_needed, v_curr_stock - v_needed, NEW.order_id);
    END LOOP;
    CLOSE cur_recipe;
END //
DELIMITER ;

```

Trigger: trg_audit_price_change

```

DELIMITER //
CREATE TRIGGER trg_audit_price_change
BEFORE UPDATE ON menu_items
FOR EACH ROW
BEGIN
    IF OLD.price <> NEW.price THEN
        INSERT INTO price_audit_log (item_id, old_price, new_price, changed_at, changed_by)
        VALUES (OLD.item_id, OLD.price, NEW.price, NOW(), CURRENT_USER());
    END IF;
END //
DELIMITER ;

```